

บทที่ 4

การค้นหา Superimposed Text บนวิดีโอ

4.1 เทคนิคการค้นหา Superimposed text บนวิดีโอแบบอัตโนมัติ

จุดประสงค์หลักของการทำบทสรุปของวิดีโอ คือ การบีบอัดหัวข้อที่ไม่สำคัญทิ้งไปที่มีผลต่อการจัดเก็บไฟล์วิดีโอนั้น จะเห็นได้ชัดอย่างยิ่ง ในข่าว และ เกมกีฬา เพราะผู้ใช้นั้นจะให้ความสำคัญกับเฉพาะเหตุการณ์ที่ตนสนใจเท่านั้น โดยจะไม่ต้องการที่จะใช้เวลาไปกับการรอคอยฉากที่ตนเองอยากชม ดังนั้นการทำบทสรุปของวิดีโอ (Video summarization) นั้นจะมีผลต่อการค้นหา ซึ่งพบว่า ข้อความที่ขึ้นมานั้น ก็นับว่าเป็นการอธิบายเหตุการณ์ของวิดีโอในช่วงเวลานั้นเป็นอย่างดี เช่น เกมกีฬาฟุตบอล จะมี Superimposed text บอกคะแนนการทำประตู เหตุการณ์ต่างๆ เพื่อจะดึงดูดให้เกมกีฬานั้นดูน่าสนใจมากยิ่งขึ้น

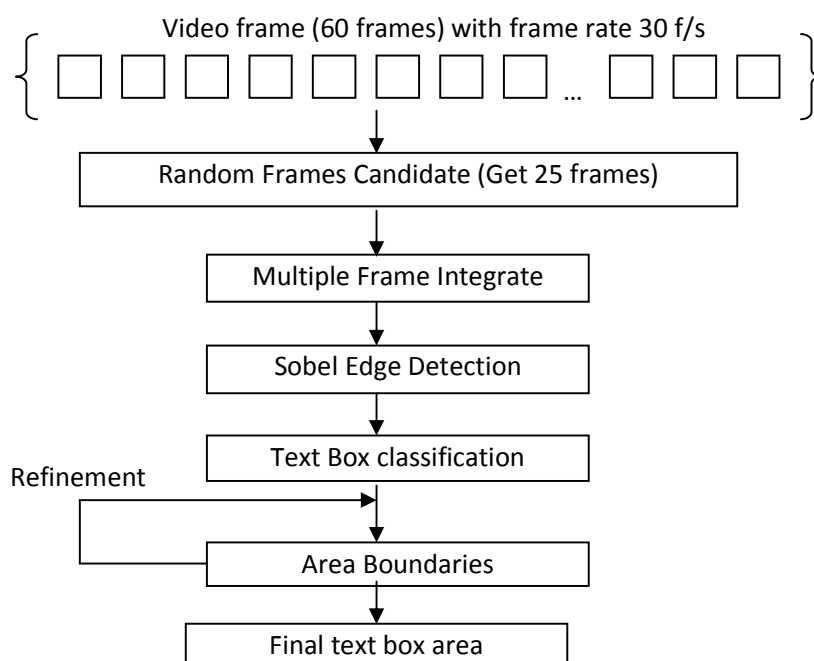


รูป 4.1 ภาพจากต่างๆ ที่มี Superimposed Text ปรากฏ

โดยในขั้นตอนการค้นหา Superimposed Text นั้น จะเริ่มจากการ ค้นหา (Detection) พิกัดตำแหน่ง (Localization) และ ตัดทอน (Extraction) ซึ่ง การค้นหานี้ จะเป็นการแบ่งอย่างคร่าวๆ ว่า พิกเซลไหนคือพิกเซลที่อยู่ในขอบข่าย การบอกพิกัดตำแหน่ง เป็นการพิจารณาเส้นขอบของตัวหนังสือ ในการตัดทอนนั้น เป็นขั้นตอนการฟิลเตอร์เอาพิกเซลพื้นหลังออก ทำให้เหลือเพียงแค่ พื้นที่ที่มีแค่ตัวหนังสือ จะสังเกตได้ว่า การตรวจหา superimposed text ในวิดีโอ นั้นจะมีสิ่งที่เป็นผลกระทบหลายอย่าง เช่น คุณภาพของวิดีโอเอง ในการส่งสัญญาณทีวี นั้นอุดมไปด้วย noise ซึ่งยากแก่การทำ image processing แม้ว่าจะใช้วิธีการลด noise ก็จะทำให้ภาพนั้น เบลอได้ , การที่มีพื้นหลัง กับ superimposed text ที่ยากจะแยกออกจากกัน , ความละเอียดต่ำ, ขนาดของตัวอักษร ซึ่งสิ่งเหล่านี้ ทำให้ยากแก่การตรวจหา superimposed text

4.1.1 ภาพรวมเทคนิค การค้นหา Superimposed Text

จากการสังเกตของผู้จัดทำ พบว่า “Superimposed text นั้นจะมีการแสดงบนวิดีโออย่างน้อย 2 วินาที” จากการสังเกตนี้ ทางผู้จัดทำได้ออกแบบ Algorithm สำหรับการค้นหา superimposed text โดยเขียนออกเป็นแผนภาพ ดังรูปที่ 4.2



รูปที่ 4.2 ขั้นตอนการค้นหา Superimposed Text

4.1.2 Random Frame Candidate

จากข้อสังเกตนั้น จะวิเคราะห์เป็นช่วง ๆ ของวิดีโอ โดยวิเคราะห์ทีละ 2 วินาที โดยมี frame rate 30 frames/sec ดังนั้น จะวิเคราะห์ทีละ 60 เฟรม จากนั้น จะผ่านกระบวนการ Random Frames Candidate เป็นการเลือกมาเฟรมแบบ Random มาเพียง 25 เฟรมมาเท่านั้น เพราะถ้าวิเคราะห์ทั้งหมด 60 เฟรม จะทำให้เปลืองทรัพยากรเครื่องโดยเปล่าประโยชน์ และ ผลที่ออกมานั้นก็ไมต่างกัน

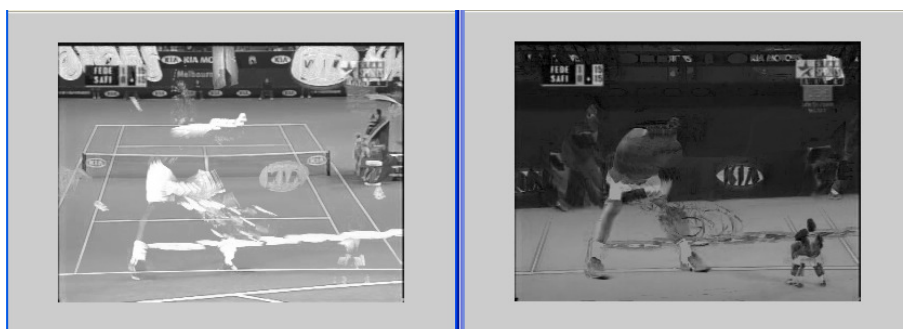
4.1.3 Multiple Frame Integration

ขั้นตอนนี้ เป็นการรวมเฟรมจากทั้งหมด 25 เฟรม เหลือเพียง 2 เฟรม ซึ่งเฟรมที่จะวิเคราะห์ นั้น จะต้องเป็นภาพ gray scale ทั้งหมด โดยทำการแปลงจากภาพสี เป็นภาพ gray scale จากนั้น นำเฟรมทั้งหมดมาคำนวณด้วยสมการ (4.1) และ (4.2) ซึ่งผลลัพธ์ ออกมาจะเหลือภาพเพียง 2 ภาพ คือ ภาพค่าสูงสุด (Max Image) และ ภาพค่าต่ำสุด (Min Image)

$$MinIMAGE_i(x, y) = \min_{j \in Ci} (p_j(x, y)) \quad (4.1)$$

$$MaxIMAGE_i(x, y) = \max_{j \in Ci} (p_j(x, y)) \quad (4.2)$$

โดยที่ Ci เป็นเฟรมที่ได้เลือกไว้ (Candidate frame = $i, \dots, i+24$) $P(x,y)$ คือ ค่าของภาพ gray scale จากสมการ นั้นแสดงให้เห็นว่า มีการเข้าถึงภาพทีละ 25 เฟรม โดยจะทำการวิเคราะห์ แต่ละพิกเซลของแต่ละเฟรม โดยจะหาค่าสูงสุด และ ค่าต่ำสุดของแต่ละตำแหน่ง แล้ว ออกมาสร้างภาพใหม่ ดังภาพที่ 4.3 ด้านล่าง



รูปที่ 4.3 ภาพค่าสูงสุด และ ภาพค่าต่ำสุด หลังจากผ่าน Multiple Frame Integration

4.1.4 Sobel Edge Detection

การหาขอบภาพ เป็นการหาขอบเขตของวัตถุภายในภาพหรือเป็นการดึงลักษณะโครงร่างที่เด่นของวัตถุออกมา การหาขอบภาพโดยใช้อุปกรณ์อันดับที่หนึ่งเป็นการแปลงเกรเดียนต์แบบไม่ต่อเนื่องบนข้อมูลภาพเชิงตัวเลข เนื่องจากการหาขอบภาพเป็นการประมวลผลแบบไม่ต่อเนื่อง ดังนั้นจึงต้องใช้อุปกรณ์ย่อยแบบไม่ต่อเนื่องตามทิศทางที่ตั้งฉากกับแกน x และแกน y ซึ่งสามารถกำหนดได้ดังนี้

$$\begin{aligned}\nabla_x g(x, y) &= g(x, y) - g(x-1, y) \\ \nabla_y g(x, y) &= g(x, y) - g(x, y-1)\end{aligned}\quad (4.3)$$

ส่วนขนาดโดยประมาณของเกรเดียนต์ $g(x, y)$ สามารถกำหนดได้ดังนี้

$$\nabla g(x, y) = (\nabla_x g(x, y))^2 + (\nabla_y g(x, y))^2 \quad (4.4)$$

เพื่อให้ง่ายต่อการคำนวณในบางครั้งจะให้ขนาดของเกรเดียนต์มีค่าประมาณขนาดของเกรเดียนต์ตามทิศทางที่ตั้งฉากกับแกน x และแกน y รวมกัน คือ

$$\nabla g(x, y) = \nabla_x g(x, y) + \nabla_y g(x, y) \quad (4.5)$$

การหาขอบภาพโดยใช้เกรเดียนต์ในทางปฏิบัติจะมีลักษณะที่แตกต่างกันไป แต่ได้เลือกวิธีของ Sobel สามารถเขียนเป็นสมการได้ดังนี้

$$|\nabla g(x, y)| = \left(\begin{aligned} & \left| \frac{g(x, y) + 2g(x, y+1) + g(x, y+2) -}{(g(x+2, y) + 2g(x+2, y+1) + g(x+2, y+2))} \right| + \\ & \left| \frac{g(x, y) + 2g(x+1, y) + g(x+2, y) -}{(g(x, y+2) + 2g(x+1, y+2) + g(x+2, y+2))} \right| \end{aligned} \right) \quad (4.6)$$

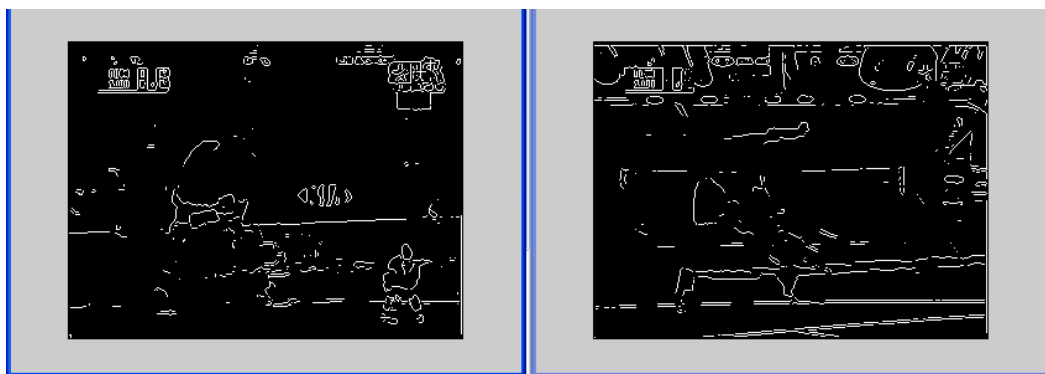
จากสมการของ (7) นั้นสามารถนำมาเขียนในรูปของวินโดว์ได้ คือ

$$W_1 = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} \quad W_2 = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (4.7)$$

การนำวินโดว์ W ของแต่ละวิธีมาหาขอบภาพ จะต้องนำวินโดว์ทุกตัวของวิธีนั้นๆ มาทำการคอนโวลูชัน (Convolution) หรือการประสานกับข้อมูลภาพ ซึ่งสามารถแสดงในรูปของสมการได้ดังนี้

$$h_k(x, y) = \sum_i \sum_j w_k(i, j)g(x+i-1, y+j-1) \quad (4.8)$$

โดย $h_k(x, y)$ คือจุดภาพ ณ ตำแหน่ง x, y ที่ผ่าน การคอนโวลูชันแล้วด้วยวินโดว์ที่ k $w_k(i, j)$ คือสมาชิกตัวที่ i, j ของวินโดว์ตัวที่ k เมื่อดำเนินการคอนโวลูชันระหว่างข้อมูลภาพกับวินโดว์ของวิธีที่จะนำมาหาขอบภาพครบทุกวินโดว์แล้วก็นำ $h_k(x, y)$ สำหรับ k ทุกตัวมาทำการเปรียบเทียบกัน ณ พิกัด x, y เดียวกันเพื่อเลือกค่าที่สูงที่สุด และนำค่าที่ได้มาเทียบกับค่าขีดเริ่มเปลี่ยน(Threshold) คือ ถ้าน้อยกว่าค่าขีดเริ่มเปลี่ยนจุดภาพนั้นก็ไม่ใช่ขอบภาพ

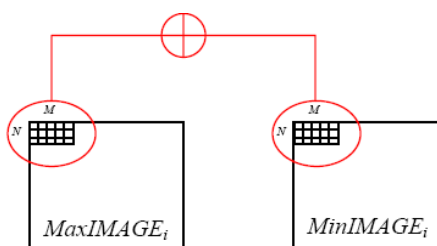


รูปที่ 4.4 ภาพหลังจากการผ่าน Sobel Edge Detection ของ MaxIMAGE & MinIMAGE

หลังจากสร้างภาพขึ้นมาได้ 2 ภาพ : MinIMAGE and MaxIMAGE จะใช้ Sobel edge detection กับภาพ MinIMAGE and MaxIMAGE และพิจารณาภาพที่มี Sobel edge point น้อยๆ จะเลือก Sobel edge operator 4 ทิศทางในการกำหนดขอบ ถ้าค่า maximal gradient ณ จุดซึ่งมีค่ามากกว่าค่า threshold มันจะสามารถพิจารณา ณ จุดขอบนั้น ในรูป 4.4 เป็นการแสดงตัวอย่างหาขอบด้วยวิธีของ Sobel

4.1.5 Text Box Classification

ในขั้นตอนการจัดแบ่งประเภทของข้อความ จะใช้ขนาด $M*N$ ($20*10$ pixel) เป็นวินโดว์ สำหรับตรวจสอบรายละเอียดของการรวมกันของภาพ และจัดกลุ่มในแต่ละวินโดว์ว่าเป็น text หรือ non-text ซึ่งในการกำหนดค่า M และ N นั้นจะกำหนดให้ค่า $M > N$ เพราะว่า ขนาดของวิดีโอ นั้นจะมีค่า แนวนอนมากกว่าเสมอ ซึ่งถ้าค่าความเข้มของขอบในวินโดว์ (Edge density) มีขนาดใหญ่กว่าค่า threshold ที่กำหนดไว้ล่วงหน้าซึ่งจะมีการแบ่งประเภทของ text มีฉะนั้นจะมีการแบ่งประเภทของ non-text ซึ่งในการทดลองค่า threshold ที่ใช้คือค่า 0.70 โดยจะไม่ย้ายตำแหน่งของวินโดว์ทุก ๆ หนึ่งพิกเซล เพราะว่ามันจะสิ้นเปลืองเวลามากและไม่จำเป็นจะต้องพิจารณาการแลกเปลี่ยนค่าระหว่างความแม่นยำ กับความเร็ว เลื่อนวินโดว์ในแนวตั้ง 10 พิกเซล และ แนวนอน 5 พิกเซล ณ เวลานั้นๆ



รูปที่ 4.5 วิธีการ Text box classification

จากภาพที่ 4.5 นั้นจะมีการใช้เชื่อมความสัมพันธ์ของ MaxIMAGE และ MinIMAGE ด้วยการกระทำแบบ AND ซึ่งจะช่วยลดข้อผิดพลาดในแต่ละวินโดว์ด้วย และ อีกทั้งยังเป็นการรวมภาพเข้าด้วยกันให้เหลือเพียงภาพเดียว ซึ่งผลลัพธ์ออกมา ได้ดังภาพ 4.6



รูปที่ 4.6 ผลลัพธ์จากการทำ Text box classification

4.1.6 Area boundaries

เป็นการขยายขนาดของพื้นที่ ที่กำหนดเป็น text เพราะว่า พื้นที่ที่กำหนดได้นั้น ที่ขอบ จะมีไม่สามารถตรวจหาได้ เพราะ พื้นหลังนั้นมีสีที่ใกล้เคียงกับตัวพื้นที่ที่เป็น text มาก ทำให้ต้องขยาย ขอบ แล้วทำกับ กำหนดใส่เส้นแสดงว่า เป็นพื้นที่ ของ superimposed text ดังภาพที่ 4.7

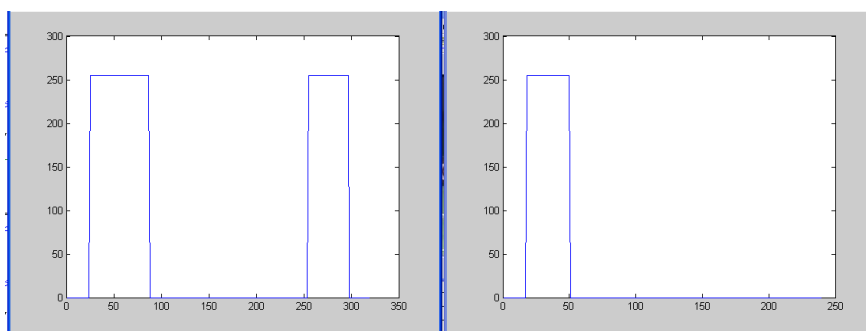


รูปที่ 4.7 ผลลัพธ์จากการทำ Area boundaries



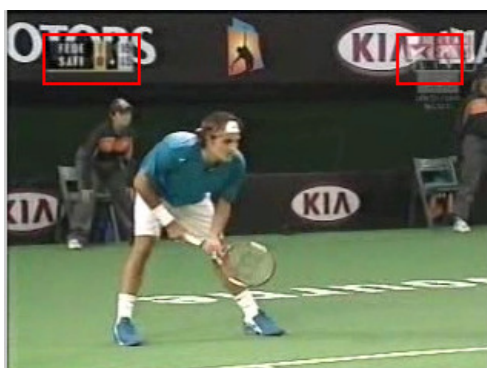
รูปที่ 4.8 ผลลัพธ์จากการทำ Area boundaries ในวิดีโออื่นๆ

ขอบเขตของพื้นที่ที่เป็น text นั้น ไม่ใช่ภาพสี่เหลี่ยม ดังนั้น จะต้องทำการ Refinement หรือ ปรับปรุงกรอบของพื้นที่ที่ต้องการ โดยใช้วิธีการ Projection ดังกราฟที่ 4.9



รูปที่ 4.9 กราฟที่เกิดจากการทำ projection

โดยจะทำการฉายในแนวตั้ง และ แนวนอน โดยนำค่าแต่ละพิกเซลมาบวกกันในแต่ละแกน ทำให้สามารถหาจุดตัดของ ดังภาพที่ 4.10



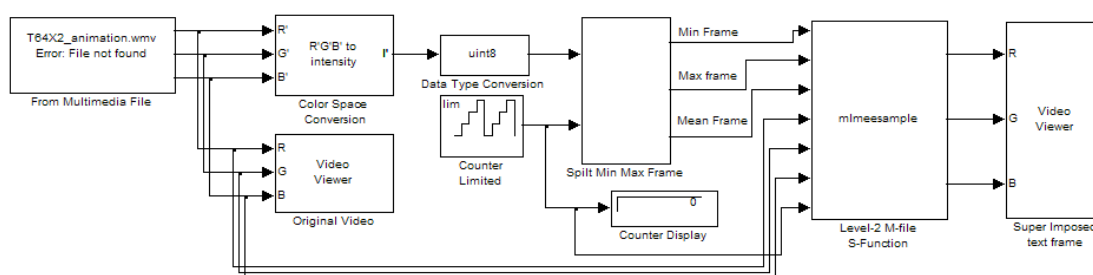
รูปที่ 4.10 text box หลังจากการทำ refinement

4.2 การจำลองอัลกอริทึมบน Simulink ของ MATLAB

ในหัวข้อที่ผ่านมาได้มีอธิบายหลักการค้นหา Superimposed text และได้แสดงผลลัพธ์ของการทำงานในลำดับขั้นต่างๆ ไปแล้ว ทางผู้พัฒนาจะนำผลลัพธ์ต่างๆ เหล่านี้ มาแสดงการจำลองการทำงานใน Simulink ซึ่งจะสามารถแสดงผลลัพธ์ในระยะเวลาที่ต้องการทดสอบได้ จากการออกแบบและทดสอบผลในส่วน Simulink นี้ จะสามารถนำไปออกแบบระบบฮาร์ดแวร์บน FPGA ด้วย Xilinx System Generator

จะทำการสร้างโมเดลขึ้นมาตามรูปที่ 4.11 โดยจะใช้โปรแกรม Simulink ซึ่งเป็นส่วนหนึ่งของโปรแกรม MATLAB เข้ามาทำการออกแบบระบบนี้

ในการออกแบบนั้นได้มีการตั้งค่า sampling time ของทั้งระบบไว้ที่ 1/25 หรือ 25 ครั้งต่อวินาที (ค่าเทียบเท่ากับจำนวนเฟรมต่อวินาทีของระบบ PAL) เพื่อให้เข้ากันกับไฟลส์วิดีโอที่นำมาทดสอบได้ (เฟรมเรต 25 เฟรมต่อวินาที) และง่ายต่อการคำนวณเวลาโดยรวม และได้ตั้งชนิดของข้อมูลที่จะใช้ส่งรับในระบบทั้งหมดให้เป็น UNSIGNED INT 8 BITS ทำให้การรับส่งค่าในระบบเป็นไปในแนวทางเดียวกัน เพื่อที่จะสามารถหลีกเลี่ยงปัญหาการขัดแย้งกันของชนิดข้อมูลที่จะเกิดขึ้นใน Simulink ได้

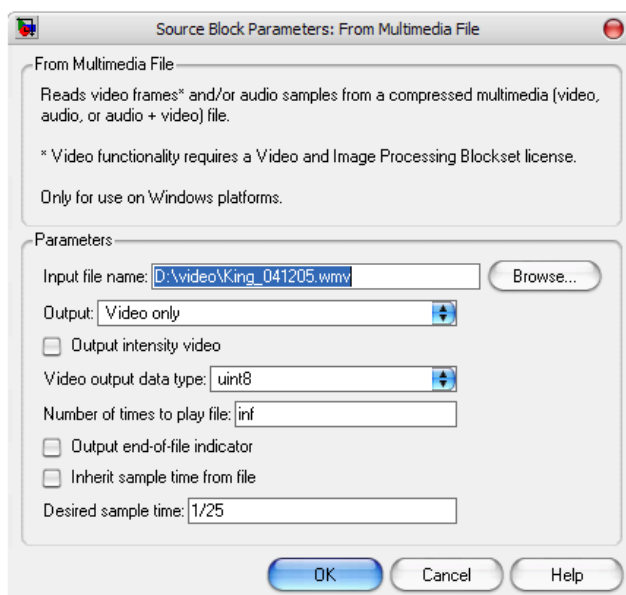


รูปที่ 4.11 Model การจำลองระบบคำหาพื้นที่ Superimposed text บน Simulink

ตารางที่ 4.1 เซตค่าพื้นฐานของโมเดลใน Simulink

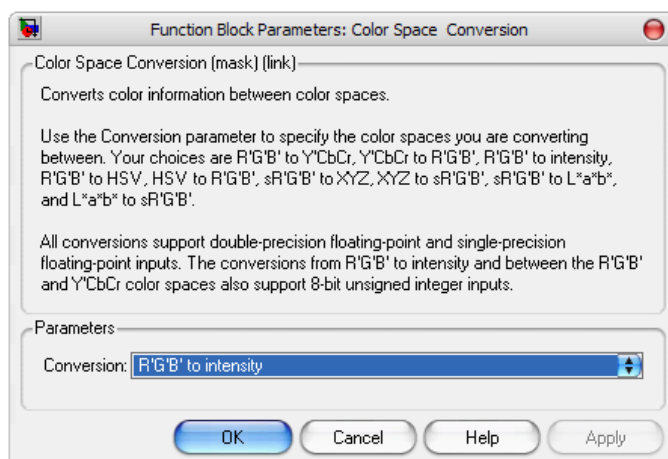
Block	Library	Quantity
From Multimedia File	Video and Image Processing Blockset / Sources	1
Color Space Conversion	Video and Image Processing Blockset / Conversions	1
Video Viewer	Video and Image Processing Blockset / Sinks	2
Data Type Conversion	Simulink / Signal Attributes	1
Counter Limited	Simulink / Sources	1
In1	Simulink / Ports & Subsystems	4
Out1	Simulink / Ports & Subsystems	6
Memory	Simulink / Discrete	50
Display	Simulink / Sinks	1
Subsystem	Simulink / Ports & Subsystems	2
Level-2 M-file S-function	Simulink / User-Defined Functions	1

ขั้นตอนถัดมาเลือกไฟล์ ที่ต้องการในบล็อกพารามิเตอร์ From Multimedia File ในช่อง Input file name โดยสามารถเลือกไฟล์วิดีโอใดๆที่ตัวเครื่องสนับสนุน ออกมาคำนวณใน Simulink ได้ ทางผู้พัฒนานั้นได้ทำการลงโปรแกรม “K-Lite Codec Pack” ซึ่งเป็นโปรแกรมที่รวบรวม encoder สำหรับการรับชมไฟล์วิดีโอรูปแบบต่างๆที่มีอยู่ในอินเทอร์เน็ต หลังจากนั้น ให้เลือกช่อง Output: เป็น Video only เพราะต้องการเฉพาะข้อมูลภาพมาใช้ในการคำนวณเท่านั้น จากนั้นให้ตั้งค่า Video output data type: เป็น UINT8 หรือก็คือ unsigned integer 8 bit ที่ได้กล่าวไว้ในตอนต้นของบทนั่นเอง ต่อมาในช่อง Number of times to play file: ในตั้งไว้เป็น inf (ไม่มีที่สิ้นสุด) และในช่องสุดท้าย Desired sample time: ให้ตั้งเป็น 1/25 ดังภาพที่ 4.12



รูปที่ 4.12 แสดงการตั้งค่าเริ่มต้นของ Block From Multimedia File

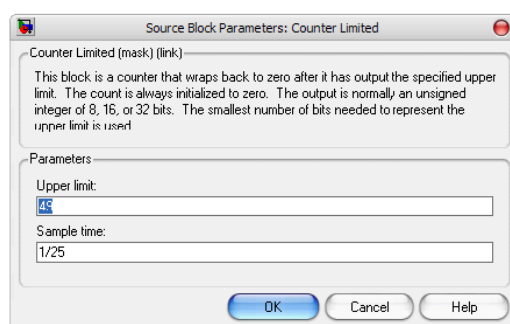
เมื่อเตรียมค่าไฟล์วิดีโอที่ต้องการจะทดสอบพร้อมแล้ว ต่อไปจะเป็นส่วนแปลงค่าของไฟล์วิดีโอ ที่ตอนนี้ยังแสดงอยู่ในรูปแบบ RGB อยู่ ให้กลายมาเป็นรูปแบบ grayscale โดยใช้บล็อก Color Space Conversion โดยให้ตั้งค่าในพารามิเตอร์เป็น R'G'B to intensity ตามรูปที่ 4.13



รูปที่ 4.13 แสดงการตั้งค่าเริ่มต้นของ Block Color Space Conversion

เมื่อได้ค่าของไฟลัวิตีไอออกมาเป็น grayscale แล้ว นำไปเข้า subsystem “Split Min Max Frame” ในพอร์ตอินพุตที่ 1 ส่วนในพอร์ตอินพุตที่ 2 จะรับค่าจำลองเวลาขึ้นมา ในการทำ superimposed text นั้น ทฤษฎีได้ตั้งเวลาที่เหมาะสมของการที่ super imposed text จะปรากฏบนไฟลัวิตี 2 วินาที ในขณะที่ได้กำหนดให้ไฟลัวิตีไอส่งข้อมูลออกมาทีละ 25 เฟรมต่อวินาที ดังนั้น ถ้าต้องการจะให้มีการทำงานในทุกๆ 2 วินาทีแล้ว จำเป็นจะต้องตั้งค่าไว้ที่ 2×25 เฟรม นั่นคือ 50 เฟรม ซึ่งจะใช้บล็อก Counter Limited เป็นตัวคอยนับจำนวนเฟรมไปเรื่อยๆ จนถึง 50 เฟรม จากนั้นจึงเริ่มต้นนับใหม่ ทำให้ได้รอบการทำงานในทุกๆ 2 วินาทีขึ้นมา จากบล็อก Counter Limited ให้ตั้งค่าพารามิเตอร์ไว้ดังนี้

- Upper limit: ให้ตั้งไว้เท่ากับ 49 (เพราะบล็อกนี้เริ่มนับขึ้นมาจากค่าศูนย์)
- Sample time: ให้ตั้งค่าไว้เท่ากับ 1/25 เหมือนกับบล็อกอื่นๆ

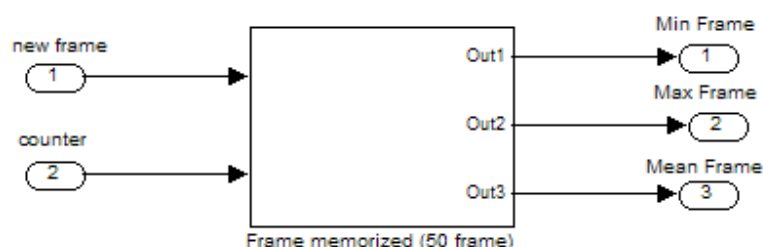


รูปที่ 4.14 แสดงการตั้งค่าเริ่มต้นของ block Counter Limited

จากนั้นให้สร้าง subsystem “Split Min Max Frame” ขึ้นมาโดยให้มีพอร์ตอินพุตเข้ามา 2 พอร์ต และมีพอร์ตเอาต์พุตออกมา 3 พอร์ต โดยกำหนดให้แต่ละพอร์ตรับค่าเข้ามาดังนี้

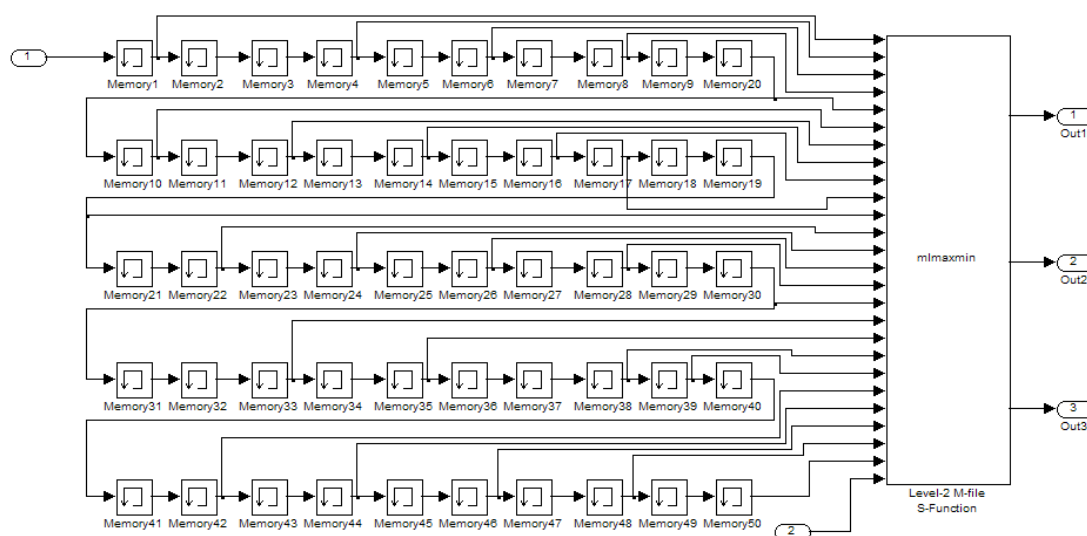
- พอร์ตอินพุต 1 : รับค่าเฟรมใหม่ที่มาจากไฟส์วิดีโอซึ่งได้แปลงเป็นสีแบบ grayscale ไว้
- พอร์ตอินพุต 2 : รับค่าจาก Counter Limited เข้ามาเพื่อคอยเช็คเวลาที่จะการคำนวณโดยรวม
- พอร์ตเอาต์พุต 1 : นำผลลัพธ์ของการรวมภาพด้วยการหา MinFRAME ออกมา
- พอร์ตเอาต์พุต 2 : นำผลลัพธ์ของการรวมภาพด้วยการหา MaxFRAME ออกมา
- พอร์ตเอาต์พุต 3 : นำผลลัพธ์ของการรวมภาพด้วยการหา MeanFRAME ออกมา

ให้สร้าง subsystem เข้าไปอีกชั้นหนึ่งใน subsystem ชั้นนี้ ชื่อ “Frame memorized (50 frame)” โดยสร้างให้มีพอร์ตอินพุตและเอาต์พุตจำนวนเท่ากับ subsystem “Split Min Max Frame” เพื่อความรวดเร็วในการแก้ไขเมื่อเกิดมีข้อผิดพลาดขึ้นมาใน subsystem “Frame memorized” ดังที่ได้แสดงไว้ในรูปที่ 4.11



รูปที่ 4.15 model sub-layer ย่อยของบล็อก Split Min/Max Frame

จากนั้นเข้าไปสู่ sub-layer “Frame memorized (50 frame)” ให้จัด model เป็นไปตามรูปที่ 3 โดยใช้บล็อก Memory ทั้งหมด 50 บล็อกเพื่อเก็บเฟรม “ทุกเฟรม” ในรอบ 2 วินาทีที่ออกมาจากไฟส์วิดีโอ จากนั้นเลือกเฟรมออกมาจากกลุ่มนี้ 25 เฟรม (ตามทฤษฎีควรจะใช่วิธีสุ่มเลือก แต่ผู้จัดทำพบว่าเป็นการสิ้นเปลืองเวลาการคำนวณของเครื่องมากเกินไป จึงตัดสินใจวิธีจิ้มเลือกออกมา 25 เฟรมแทน) นำมาเข้า S-function ที่ชื่อว่า mlmaxmin โดยคุณสมบัติของฟังก์ชันนี้มีดังนี้



รูปที่ 4.16 model sub-layer ย่อยของบล็อก Frame memorized (50 frame)

มีพอร์ตอินพุตทั้งหมด 26 พอร์ต โดยรับค่าเฟรม ณ เวลาต่างๆ 25 พอร์ต และ รับค่า counter อีก 1 พอร์ต มีพอร์ตเอาต์พุตทั้งหมด 3 พอร์ต โดยนำค่า min max mean ออกไปในแต่ละพอร์ตตามลำดับ โดย s-function นี้ทำงานตาม pseudo-code ดังต่อไปนี้

```

if InputPort(26).Data = 49
    for i = 1:25
        maxframe = max(maxframe , InputPort(i).Data)
        minframe = min(minframe , InputPort(i).Data)
        multimatrixmean(:,i) = InputPort(i).Data
    end
    for m = 1:res_m
        for n = 1:res_n
            meanframe(m,n)= mean(multimatrixmean(m,n,:))
        end
    end
    OutputPort(1).Data = minframe
    OutputPort(2).Data = maxframe
    OutputPort(3).Data = meanframe
end

```

หลังจากนั้น นำค่าเอาต์พุตออกไปยังตามพอร์ตที่ได้ตั้งไว้ในโมเดล

ในกระบวนการ superimposed text ตามที่ได้กล่าวไว้ จะยังเหลือการคำนวณ sobel edge detection, text box classification, area boundaries สามารถคำนวณกระบวนการทั้งสามนี้พร้อมกันในบล็อก S-function “mlmeesample” โดยนำค่าทั้งสามที่ได้จากข้อที่แล้ว ไปเข้าสู่ S-function “mlmeesample” ซึ่งมีคุณสมบัติการทำงานดังต่อไปนี้

มีพอร์ตอินพุตทั้งหมด 7 พอร์ต โดยรับค่าภาพสูงสุด ภาพต่ำสุด ภาพเฉลี่ย อย่างละพอร์ตเป็น 3 พอร์ต และค่า RGB จากไฟลัวิตีโอตังต้นอีก 3 พอร์ต และค่า counter อีก 1 พอร์ตและมีพอร์ตเอาต์พุตทั้งหมด 3 พอร์ต โดยนำค่า RGB ของเฟรม area boundaries โดย s-function นี้ทำงานตาม pseudo-code ดังต่อไปนี้

```

if InputPort(7).Data == 49
    % do a sobel edge detection
    edgemax = sobeledgedetection(InputPort(1).Data)
    edgemin = sobeledgedetection (InputPort(2).Data)
    edgemean = sobeledgedetection (InputPort(3).Data)

    % do text box classification
    textboxred = settextbox(edgemax,edgemin, edgemean,InputPort(4).Data)
    % make rectangle
    recttextboxred = makerectangle(textboxred)

    finalframed = max(InputPort(4).Data, recttextboxred)
    finalframegreen = block.InputPort(5).Data
    finalframeblue = block.InputPort(6).Data

    OutputPort(1).Data = finalframed
    OutputPort(2).Data = finalframegreen
    OutputPort(3).Data = finalframeblue
end

```

จาก pseudo-code ข้างต้น แสดงการทำงานของ mlmeesample ไว้อย่างคร่าวๆ หลังเสร็จสิ้นการโปรแกรมในส่วนนี้แล้ว นำเอาต์พุตออกไปยังบล็อก Video Viewer เพื่อแสดงผลลัพธ์สุดท้ายของการจำลองระบบตรวจสอบหาพื้นที่ superimposed text นี้

4.3 การพัฒนาอัลกอริทึมบนบอร์ดทดลอง Virtex-II Pro

ก่อนอื่นจะอธิบายถึงบอร์ดทดลอง XUP Virtex-II Pro Development System ตัวนี้เป็นรุ่นที่ทางบริษัท Digilent ได้ทำการออกแบบขึ้นมาเพื่อเจาะกลุ่มตลาดระดับนักศึกษามหาวิทยาลัยที่สนใจศึกษาเทคโนโลยี FPGA เพื่อนำไปใช้งานในระบบที่ต้องการความเร็วในการคำนวณสูง และด้วยความยืดหยุ่นของ component ที่ประกอบมาให้กับตัวบอร์ด จึงเหมาะสมต่อนักศึกษาที่สนใจในงานด้านนี้เป็นอย่างมาก แต่เนื่องด้วยบอร์ดตัวนี้นั้นยังคงมีข้อมูลการใช้งาน รวมถึงนักวิจัยในประเทศไทยที่ได้ทำการพัฒนามีอยู่น้อยมาก ซึ่งนับว่าเป็นอุปสรรคอย่างใหญ่หลวงสำหรับโครงการนี้

การทดสอบบอร์ดการทดลองเบื้องต้น

1. Xilinx University Virtex II Pro Development Board
2. DDR SDRAM ขนาด 256~512MB มีประโยชน์มากสำหรับงานคำนวณที่ต้องใช้พื้นที่เก็บข้อมูลที่ได้จากการคำนวณในปริมาณมาก อย่างเช่น Image Data, Video Data เป็นต้น เพราะภาพที่จะเอามาคำนวณนั้น Block-Ram ที่มีมาให้ของตัวชิป Virtex-II Pro (ขนาด 128K) ไม่สามารถรองรับได้พอกับความต้องการในโครงการนี้ได้
3. สาย USB ที่มาพร้อมกับบอร์ด (หรือจะเป็น Parallel Port ซึ่งสามารถนำมาใช้ได้เหมือนกัน แต่ในที่นี้จะใช้สาย USB ที่ให้มากับตัวบอร์ดทดลอง)
4. หม้อแปลงแรงดันไฟฟ้า
5. สาย RS-232 หรือที่รู้จักกันในนามว่า Serial Port นั้นเอง (ในโครงการนี้ได้นำ USB 2 Serial เพื่อแปลงหัวจาก Serial พอร์ตมาเป็น USB แทนน่าจะเหมาะสำหรับคนที่จะใช้ Notebook เป็นเครื่องทำงาน)
6. เครื่อง Host PC 1 เครื่อง
7. Device อื่นๆ สำหรับการทดสอบด้วย Build-In Self Test

วิธีติดตั้งโปรแกรมและข้อเสนอแนะ

ทำการ install โปรแกรม Xilinx ISE Foundation, Xilinx EDK 8.1 (ต้องเป็นซอฟต์แวร์ของ Xilinx เวอร์ชันที่ตรงกันกับบอร์ดทดลองนี้) เมื่อติดตั้งโปรแกรมสำเร็จ จากนั้นก็ทำการรีบูตเครื่อง

แล้วทำการลง Driver สาย USB กับตัวเครื่อง โดยเสียบสาย USB เข้าตัวเครื่อง PC และตัวบอร์ด เปิดสวิตช์บอร์ด ระบบจะทำการ Initialize ตัวมันเอง ให้ตอบตกลงตามระบบที่แนะนำ ซึ่งเมื่อติดตั้งเสร็จเรียบร้อยแล้ว ก็สามารถเข้าไปเช็คใน Device Manager ถ้าทำการติดตั้งได้อย่างถูกต้อง จะมีฮาร์ดแวร์ปรากฏในลิสต์คือ Programming cables – Xilinx Platform Cable USB ขึ้นมาดังรูป (4.17)



รูปที่ 4.17 เมื่อสามารถลง Driver Programming Cables สำหรับบอร์ดทดลองได้อย่างถูกต้อง

สำหรับกรณีที่มีปัญหาลงไม่ได้ อาจมีสาเหตุมาจากการลงไฟล์ .sys แบบผิดวิธี ทางผู้พัฒนาขอแนะนำ ให้ทำการลบโปรแกรม Xilinx ISE ออกไปเสียก่อน แล้วตามหา Driver ชื่อ .sys ในโฟลเดอร์ Windows/System32/Driver ในไดรฟ์ที่ท่านได้ลง OS Windows ไว้แล้วทำการลบไฟล์นั้นออกไป แล้วจากนั้นก็ย้อนกลับมาสู่ขั้นตอนการติดตั้งใหม่ หากไม่ใช้วิธีนี้ ก็ไม่สามารถที่จะติดตั้งโปรแกรมใหม่ได้อีกครั้ง อาจจะต้องมีการทำ Format เครื่องใหม่ก่อนลงโปรแกรม

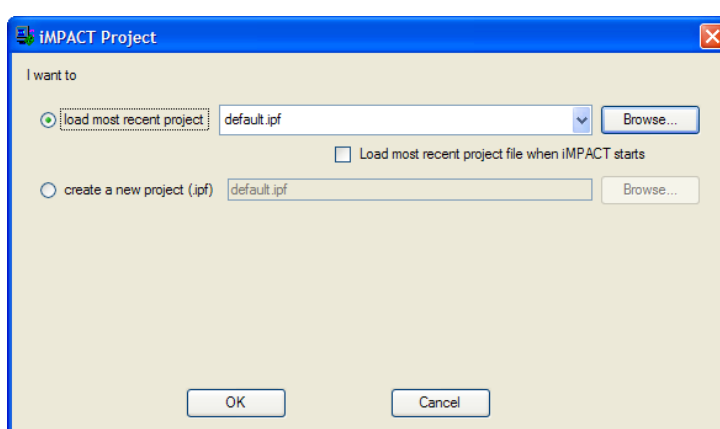
ทดสอบการใช้งานบอร์ดทดลอง

สำหรับสิ่งแรกที่ใช้งานควรจะต้องรู้เกี่ยวกับบอร์ดทดลองตัวนี้คือ “ความสามารถของบอร์ดทดลอง Virtex-II Pro” ดังนั้นจึงต้องมาทำการทดสอบก่อนว่าบอร์ดของนี้มี feature ใช้งานที่ครบหรือไม่ ภายในตัวบอร์ด XUP2VP นี้มีระบบทดสอบตัวเองที่เรียกว่า Built-In Self Test โดยสามารถดาวน์โหลดได้จาก website ของ Xilinx ในส่วนของ University Program / Hardware Section ที่ <http://www.xilinx.com/univ/xupv2p.html> โดยในเว็บไซค์นี้มีข้อมูลพื้นฐานเบื้องต้นการใช้งานบอร์ดทดลอง XUPV2P หรือสามารถค้นหาได้จากใน CD ประกอบที่ให้มากับตัวบอร์ดทดลอง

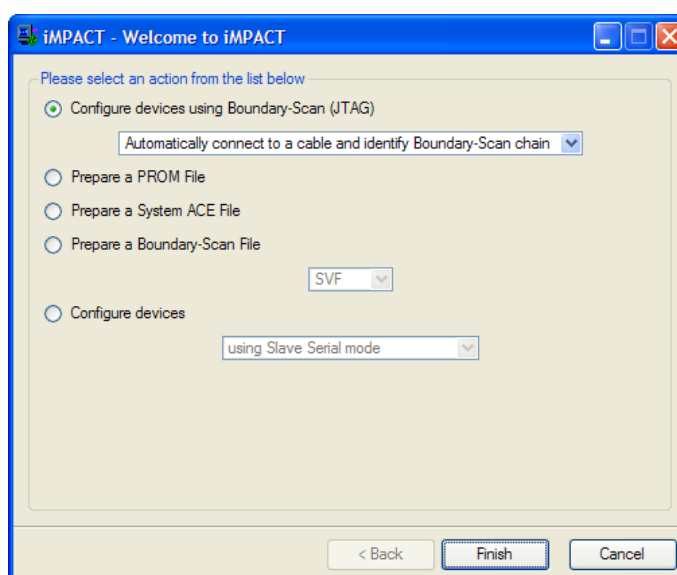
ภายในไฟล์ที่ได้ทำการดาวน์โหลดในข้างต้น จะมีโปรแกรม iMPACT โปรแกรมสำหรับการ download file ลงสู่ตัวบอร์ดทดลอง FPGA ผ่าน Programming Cables ประเภท Parallel กับ

USB หรืออาจจะให้ FPGA โหลดจาก System ACE โปรแกรมนี้ก็จะแปลง stream ไปลงสู่ compact flash ได้ด้วยเช่นกัน

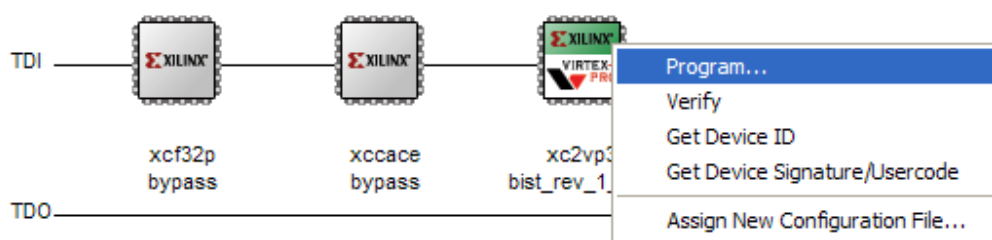
เมื่อเปิดโปรแกรมขึ้นมาแล้ว โปรแกรมจะขึ้นภาพดังต่อไปนี้ขึ้นมา ให้ทำการเลือกไปที่ create a new project (.ipf) แล้วกด OK แล้วจะขึ้นหน้า Welcome to iMPACT ดังรูปที่ 4.18 ให้เลือกที่จะให้โปรแกรมทำงานในรูปแบบไหน สำหรับตอนนี้ต้องการดาวน์โหลดไฟล์ Bitstream ลงสู่ตัวบอร์ดทดลอง ให้เลือก Configure devices using Boundary-Scan (JTAG) แล้วกด Finish



รูปที่ 4.18 Dialog Box แรกเมื่อเปิดโปรแกรม Impact



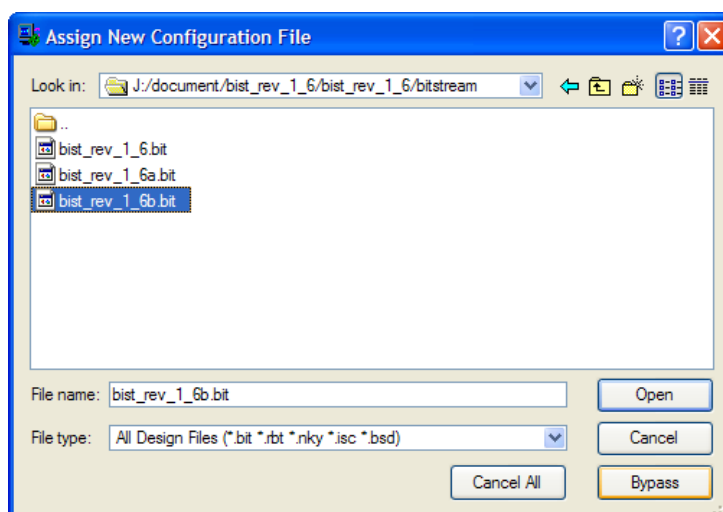
รูปที่ 4.19 หน้า Welcome to iMPACT เพื่อเลือกการใช้งานที่ต้องการจากโปรแกรม



รูปที่ 4.20 เมื่อโปรแกรมทำการ initialize chain เสร็จแล้ว

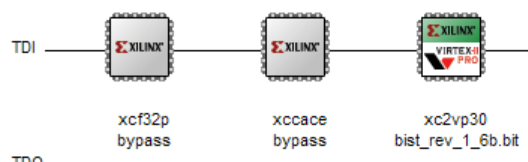
โปรแกรมจะทำการ auto initialize chain ขึ้นมา ซึ่งจะแสดงให้เห็นชิป 3 ตัวคือ xcf32p bypass, xccace bypass และ xc2vp30 bist_rev_1_6b.bit พร้อมกับ dialog ซึ่งให้โหลดโปรแกรม bitstream ผ่านลงไปในตัวชิป สำหรับ 2 ตัวแรกนั้น ให้ตั้งค่าพื้นฐานเป็น Bypass

ส่วนตัวที่สามนี้เป็นชิป Virtex-II Pro สำหรับบอร์ดทดลอง ในกรณีที่ไม่ขึ้นว่า xc2vp30 ให้ทำการตรวจสอบถึงสาย USB นั้นเชื่อมต่อระหว่างตัวบอร์ดกับเครื่อง ถ้าได้ทำการตรวจสอบเรียบร้อยแล้ว ลองทำการ Initialize chain ใหม่อีกครั้ง ถ้ายังคงเกิดปัญหาในลักษณะเดิมอยู่นั้น แสดงว่า ได้ทำการติดตั้งโปรแกรม ISE ผิดรุ่น ในกรณีที่ต้องการติดตั้ง ISE webpack ร่วมด้วยนั้น ทางผู้พัฒนา แนะนำให้ติดตั้ง ISE Foundation เสียก่อนเพื่อป้องกันการเกิดปัญหาดังที่กล่าวมา



รูปที่ 4.21 ให้เลือกไฟล์ .bit เพื่อนำมาดาวน์โหลดสู่บอร์ดทดลอง

จากนั้นทำการค้นหาไฟล์ bist_rev_1_6.bit จากในไฟล์ BIST ที่ Download มา แล้วกด OK จากนั้น ให้คลิกขวาที่ตัวชิป xc2vp30 แล้วเลือก Program ดังรูปที่ 4.20 ระบบจะทำการ Download โปรแกรมลงสู่บอร์ดให้ จากนั้นจะมี Dialog Box ขึ้นมา ถามการ Verify จากผู้ใช้งาน หลังจากนั้นระบบก็จะทำการเบิร์นลง บอร์ด โปรแกรมจะแสดงผลพัธดังภาพที่ 4.22



Program Succeeded

รูปที่ 4.22 เมื่อทำการดาวน์โหลดไฟล์ .bit ลงสู่ตัวบอร์ดทดลองได้สำเร็จ

ภายใน Build-In Self Test ที่ได้ทำการเบิร์นลงไปนั้น เป็นตัวทดสอบประสิทธิภาพของบอร์ด XUPV2P ว่าสามารถทำอะไรได้บ้าง โดยมี feature ในระบบ Build in Self Test นี้จะรองรับการทดสอบบอร์ดทดลองใน 2 รูปแบบ คือ Non Processor Based test Design กับ Processor Based test design โดยที่แบบแรกนั้นครอบคลุมการทำงานของบอร์ดดังนี้

- 1) Power Supply และ RESET generation
- 2) 100MHz system clock, 75MHz MGT clock, 32MHz SystemACE controller clock
- 3) LEDs
- 4) Push buttons กับ DIP switches
- 5) การแสดงออกทาง VGA มอนิเตอร์
- 6) Audio CODEC beep tone passthrough and power amplifier
- 7) RS232 serial port interface
- 8) PS/2 serial port interface
- 9) Silicon Serial Number
- 10) พอร์ตเพิ่มเติม

11) Configuration Source and mode selection, Platform FLASH, USB, PC4 and SystemACE Compact FLASH

สำหรับระบบที่จำเป็นต้องใช้ Processor ในการควบคุมมีได้แก่

- 1) SystemACE controller processor interface
- 2) Audio CODEC PCM data path
- 3) 10/100 Ethernet interface
- 4) MGT ports
- 5) DDR SDRAM module and Serial Presence Detect PROM

เมื่อได้ติดตั้ง Serial Port เข้ากับตัวเครื่องเป็นที่เรียบร้อยแล้ว ให้เปิดโปรแกรม Terminal ขึ้นมา (ใน Window-> Hyper Terminal -> Accessories/Connection) เลือกสร้างงานชุดใหม่ ขึ้นมา ตั้งชื่อว่า “9600Test” เลือก Connect Using เป็น ตำแหน่งที่ติดตั้ง Serial Port (ในกรณีของ Usb2Serial เลือกค่าเป็น Com 3 แต่ในกรณีที่พัฒนาด้วย Serial Port โดยตรงให้กำหนดค่าเป็น Com 1) แล้วตั้งค่าดังต่อไปนี้

Bits per Second= 9600

Data Bits = 8

Parity = None

Stop Bits = 1

Flow Control = None

ต่อไปทำการเปิด Switch Reset บนบอร์ดทดลองณ บริเวณตรงมุมขวาของบอร์ดทดลองกดเพียง 1 ครั้ง (ในกรณีที่ทำการกดค้าง 1~2 วินาที จะเป็นการล้างโปรแกรมของตัวบอร์ดออกไป) เมื่อกด Reset แล้วทีนี้จะเห็น LED ติดไฟขึ้นซึ่งแสดงว่าถึงตัวโปรแกรม Initial ที่ทาง Xilinx ใส่เข้ามาแสดงการทดสอบตัว LED ในกรณีที่ถ้าไม่มีข้อผิดพลาดระหว่างการทดสอบ ข้อความดังรูป 4.23 ใน terminal

XUP-V2Pro BuiltIn Self Test Main Menu Rev. 1.6 Sept. 2005

-
- 1 - Test SATA port with Aurora loopback.
 - 2 - Test Ethernet with WEB example.
 - 3 - Test AC97 audio codec.
 - 4 - Test System ACE.
 - 5 - Test DDR SDRAM.
 - 6 - Test Expansion connectors.
 - q - Quit

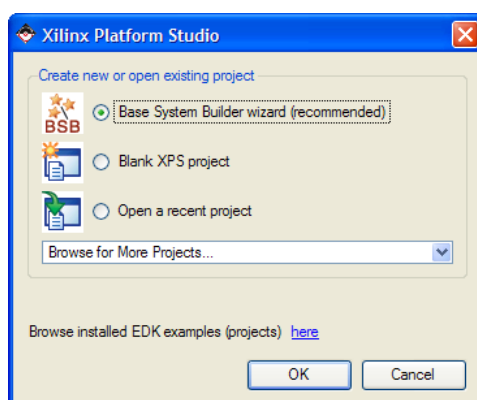
รูปที่ 4.23 หน้าจอต้อนรับของระบบ Build-In Self Test ผ่านโปรแกรม Hyper Terminal

เริ่มต้นการใช้งาน EDK 8.1

บทนี้จะเริ่มการใช้งาน Xilinx EDK แล้วครับ ขอเกริ่นเล็กน้อยก่อน EDK คือ Embedded Development Kit ที่ทาง Xilinx พัฒนาขึ้นมา โดยที่บอร์ด XUP2VP นี้มีความสามารถรองรับระบบ PowerPC กับ MicroBlaze ได้ เป็น Soft Processor Core สำหรับใช้งานใน FPGA โดยเฉพาะ

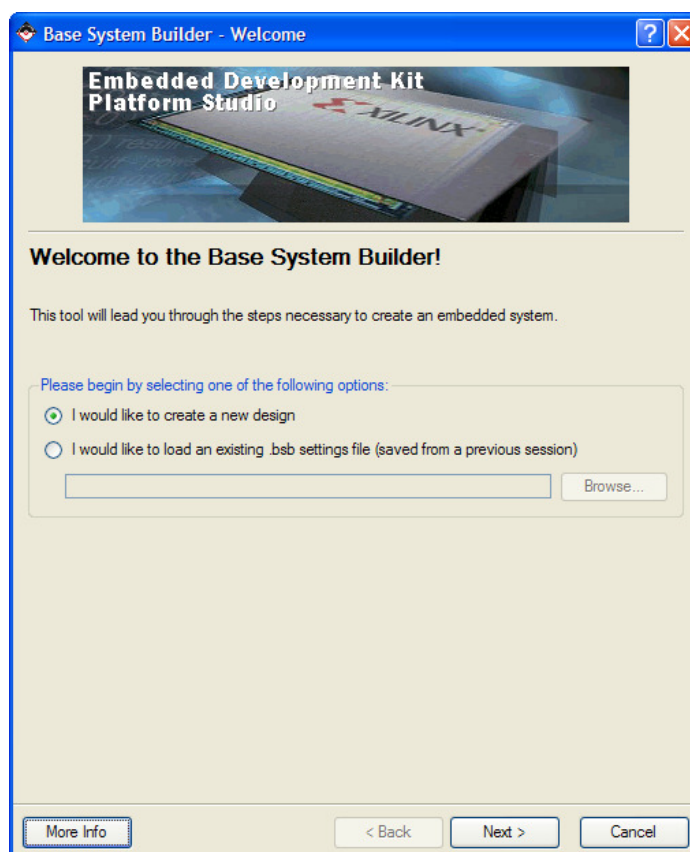
ก่อนจะพัฒนาโปรแกรมด้วยเครื่องมือ EDK นั้นๆ ต้องทำการดาวน์โหลด Package http://www.xilinx.com/univ_/XUPV2P/lib/lib_rev_1_1.zip นี้มาแล้วทำการ Unzipออกแล้วค้นหาไฟล์ชื่อ boards จากนั้นก็อปปี้ Xilinx_XUP_V2P ไปวางไว้ใน ...EDK\board\Xilinx\boards ในไฟล์ Xilinx_XUP_V2P นั้นเป็นไฟล์อธิบายลักษณะของตัวบอร์ดทดลองในโครงการนี้

เปิดโปรแกรม EDK ขึ้นมา ตัวโปรแกรมจะถามว่า ให้เลือก BSB Builder แล้ว OK จากนั้น ให้เลือก Folder ที่จะเก็บ Project นี้ ทางผู้พัฒนาแนะนำให้ตั้ง Folder ขึ้นมาใหม่สำหรับงาน โดยเฉพาะเนื่องจากใน Project XPS นี้จะมีไฟล์ย่อยอยู่เยอะอาจก่อให้เกิดความสับสนระหว่างการพัฒนาได้



รูปที่ 4.24 หน้าแรกของ Base System Builder Wizard

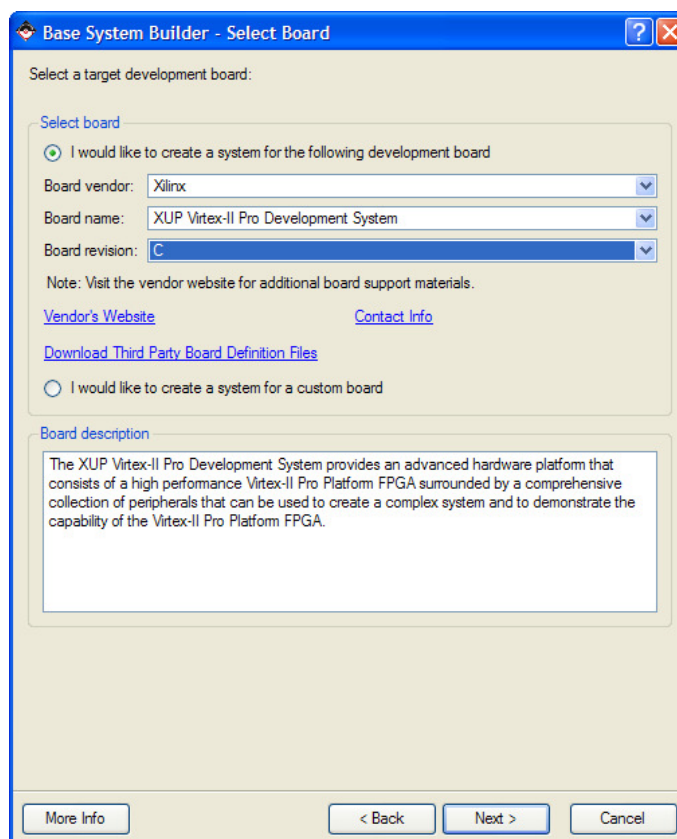
ต่อมาให้เข้าสู่ Base System Builder Wizard ในหน้าแรกนั้นให้ทำการเลือก “I would Like to create a new Design” ในส่วนอีกตัวเลือกนั้น จะเป็นการนำ bsb file ที่ setting ไว้เป็นประจำแล้วมาใช้งานใน Project ใหม่ซึ่งในขั้นตอนแรกนั้น ทางผู้พัฒนาได้สร้างโปรเจกใหม่ขึ้นมา มาก่อน



รูปที่ 4.25 หน้าจอต้อนรับผู้ใช้งานของ Base System Builder

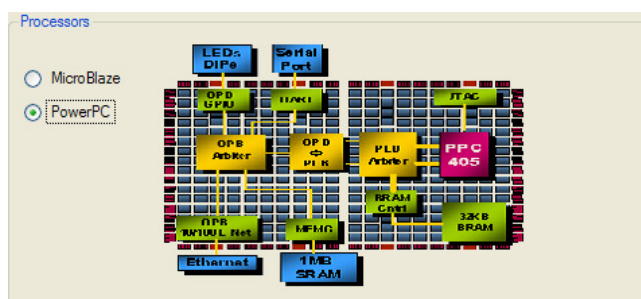
ต่อมาจะเป็นการเลือกบอร์ดทดลองที่จะใช้ทำงาน ให้เลือก “I would like to create a system for the following development board” จะเห็นว่าในส่วนของ Board Name นั้นมีบอร์ดของทาง Xilinx รุ่นต่างๆ แต่เนื่องจากได้พัฒนาบนบอร์ดรุ่น Virtex-II Pro ให้เลือก “XUP Virtex-II Pro Development Board” ที่อยู่ท้ายสุดของตัวเลือก และเลือก Board Revision เป็น “C” ในกรณีที่ไม่มีพบพบชื่อบอร์ด XUPV2P ก็แสดงว่ายังไม่ได้ก๊อปปี้ Package ของตัวบอร์ด XUPV2P ลงใน

...\EDK\board\Xilinx\boards ซึ่งให้ cancel กระบวนการ Base System Builder Wizard นี้ไปก่อนแล้ว
ไปก๊อปปี้ Package ลงในโฟลเดอร์ในขั้นตอนที่กล่าวมาในข้างต้น



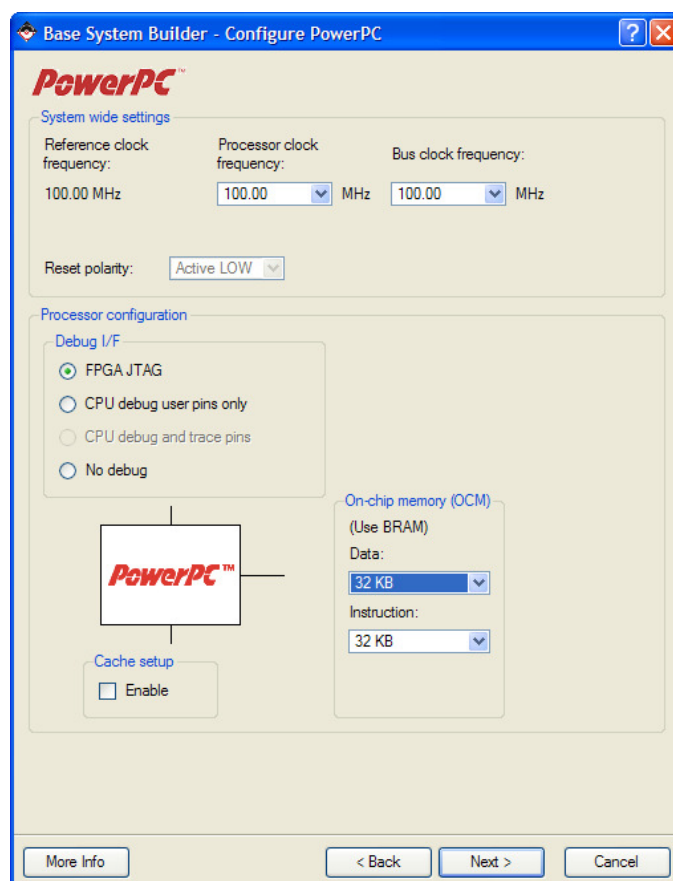
รูปที่ 4.26 เลือกบอร์ดทดลองที่ต้องการจะทำการออกแบบระบบ

ต่อมาก็ให้เลือก Processor Core ที่จะใช้โครงการ ซึ่งในที่นี้จะมาเลือกใช้บริการของ PowerPC Core ดังภาพที่ 4.27



รูปที่ 4.27 ทำการเลือก Processor Core ที่จะใช้งาน

ต่อมาให้ทำการ Configure ตัว PowerPC ซึ่งในส่วนนี้ ทางผู้พัฒนาจะอธิบายไปที่ละตัวเลือกเพราะต้องอาศัยความเข้าใจในการตั้งค่าต่างๆ โดยจะอธิบายเฉพาะส่วนที่ใช้ในโครงการนี้เท่านั้น

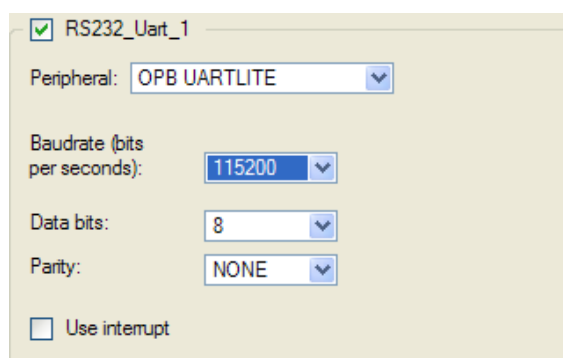


รูปที่ 4.28 หน้าจอ Configure PowerPC ในส่วนแรก

System Wide Setting จะมี Processor Clock Frequency ความถี่สัญญาณนาฬิกาสำหรับการทำงานของตัว Processor ให้ตั้งไว้ที่ 100 MHz นะครับ Bus Clock Frequency ความถี่ของระบบ Bus ในการส่งผ่านข้อมูลไปมาระหว่าง Device ต่างๆกับตัว Core Processor ทางผู้พัฒนานั้นตั้งค่าไว้ที่ 100 MHz ซึ่งค่านี้จะนำไปประยุกต์ใช้ร่วมกับ SDRAM ในภายหลัง

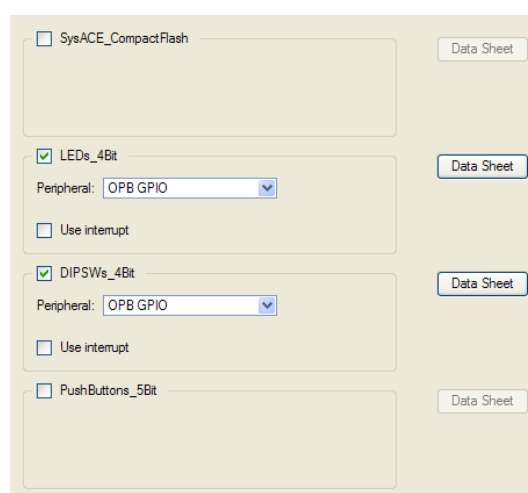
Processor Configuration สำหรับ Debug I/F ให้ตั้งค่าไปที่ FPGA JTAG หมายความว่า จะทำการ Debug ระบบผ่านทางสาย USB ส่วนค่า On-Chip Memory ซึ่งมีไว้ใช้เก็บ Data กับ Instruction ในโครงการนี้ มีความต้องการจะเก็บคำสั่งโปรแกรมและข้อมูลที่เกิดขึ้นระหว่างทำงาน

เอาไว้ในส่วนของ BlockRam ดังนั้นตั้งค่าสำหรับทั้งสองช่องไว้ที่ 32KB สำหรับ Cache ปล่อยว่างไว้ เพราะ ยังไม่ใช้งานในคำสั่งที่ต้องการ Feed ไปกลับระหว่าง Memory และตัว Processor แต่จะทำการ initialize ค่า Cache เอาจาก Software ของผู้พัฒนาได้ในภายหลังแทน



รูปที่ 4.29 ตั้งค่าสำหรับการใช้งาน Serial Port

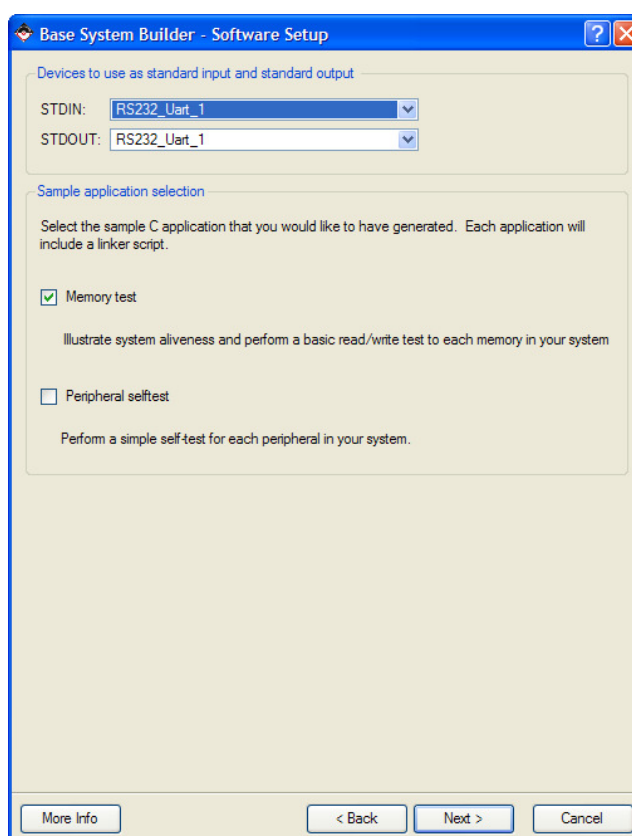
ในส่วนของ RS232_Uart_1 ให้ตั้งค่าดังต่อไปนี้ ตามรูปที่ 4.29 โดยให้ Baud rate = 115200 (ในส่วนของ Baud rate นี้ ส่วนมากจะใช้งานกันที่ 9600 แต่ทางผู้พัฒนาเลือกจะใช้ในระดับ 115200 ซึ่งเร็วกว่า 12 เท่าเพื่อที่จะมีประโยชน์ในเรื่องการ access ความเร็วที่ต้องส่งผ่านมาทาง terminal เป็นการลดปัญหาคอขวดของระบบลงไปได้มาก โดยเฉพาะในระบบที่จะต้องข้อมูลส่งค่าไปมาระหว่าง PC กับ FPGA) Data Bits = 8 Parity = None



รูปที่ 4.30 ตั้งค่าสำหรับการใช้งาน LED ขนาด 4 บิต และ DipSwitch ขนาด 4 บิต

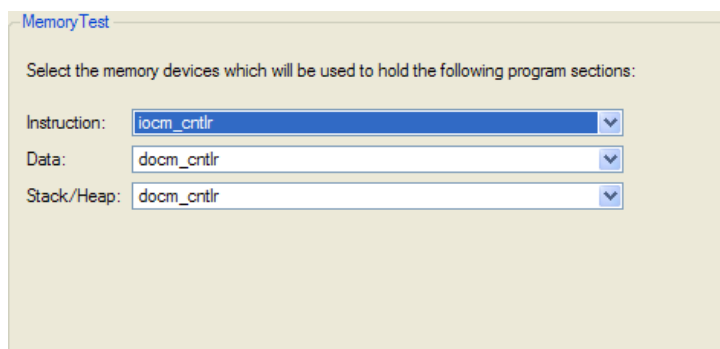
ในส่วนของ LED กับ Dip Switch ทั้งคู่ให้ตั้ง Peripheral ไว้ที่ OPB GPIO เพราะเป็น device ที่ไม่จำเป็นต้อง access ให้เร็วแต่อย่างใดนัก

ในส่วนของ Software Setup นั้น ที่ตัวเลือกของ STDIN,STDOUT ให้เลือกเป็น RS232_Uart_1 ทั้งคู่ ผู้พัฒนาต้องการส่งข้อมูลไปมาผ่านทาง Serial Port ส่วน Sample Application Selection ให้เลือกเฉพาะ memory test



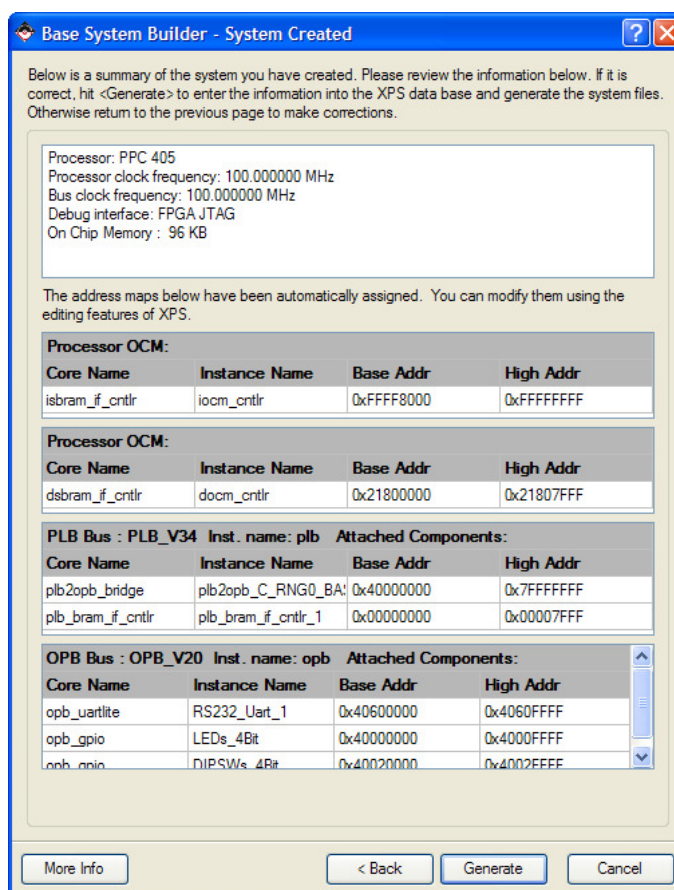
รูปที่ 4.31 ตั้งค่าในส่วนของ Software Setup

จากที่กล่าวมานั้น ผู้พัฒนาเลือกให้โปรแกรม Generate Software ชุด Memory Test ขึ้นมา ก็จะมีการให้ตั้งค่าว่าจะเก็บ Instruction, Data, Stack/Heap ไว้ที่ไหนบ้าง โดยตั้งไปว่า iocm_cntrl สำหรับเก็บส่วนของ Instruction และ docm_cntrl สำหรับเก็บข้อมูลส่วน Data กับ Stack/Heap



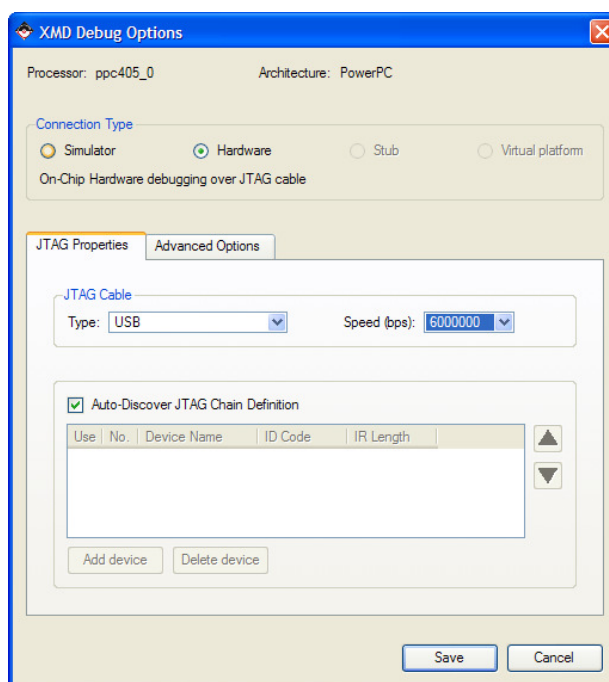
รูปที่ 4.32 ตั้งค่าระบบที่จะจัดเก็บข้อมูล Data, Instruction และ Stack/Heap ไว้

เมื่อทำการตั้งค่าทั้งหมดเรียบร้อยแล้ว โปรแกรมจะแสดงสรุปทั้งหมดของระบบที่สร้างขึ้นมาให้กด Generate เป็นอันเสร็จสิ้นการตั้งค่า Base System Builder ของโปรเจ็ค



รูปที่ 4.33 หน้าจอสรุปการตั้งค่าทั้งหมดที่เกิดขึ้นในระบบ

ขั้นตอนต่อมาทำการ Generate Hardware กับ Software เพื่อใช้ในงานของโครงการ สำหรับในส่วนของ Hardware นั้นให้เลือก Hardware -> Generate Bitstream เพื่อทำการ synthesis ระบบทาง Hardware ขึ้นมา ในขั้นตอนจะต้องใช้เวลาประมาณ 4-6 นาทีในการ Generate ระบบทั้งหมด ออกมา (ความเร็วขึ้นอยู่กับประสิทธิภาพของเครื่อง) เมื่อเสร็จแล้ว ต่อไปจะคอมไพล์ Software ตัว AppMemory ที่ได้ทำการเขียนเอาไว้ก่อนหน้านี้ เตรียมเอาไว้ โดยการพัฒนา software นั้นจะพัฒนาโดย ภาษา C ซึ่งจะต้องมีการพัฒนาโปรแกรมก่อนการ Map กับฮาร์ดแวร์ เมื่อโหลดเข้าไปแล้ว ทำการ Build Project ภายใน Software -> Build All User Application ซึ่งจะทำการสร้าง Library ขึ้นมารองรับกับ Hardware ที่ได้สร้างไปก่อนหน้านี้ เมื่อ Build เสร็จแล้ว ให้ทำการ Download ไฟล์ Bitstream ของโปรเจกต์ใหม่นี้ไปลงยังบอร์ด FPGA ที่เตรียมไว้ สำหรับการ Download แบบนี้จะเป็นการเตรียมระบบให้พร้อมไว้ก่อน แต่ยังคงขาด Software ที่จะมาทำงานในระบบ หลังจากนั้นทำการเปิดระบบ Xilinx Microprocessor Debugger (XMD) เพื่อดาวน์โหลด Software ลงสู่ตัว FPGA ให้กดปุ่ม Launch XMD หรือเลือกได้จาก Debug -> Launch XMD...



รูปที่ 4.34 ตั้งค่าสำหรับ XMD ก่อนเริ่มใช้งาน

จากนั้นระบบจะแสดง Dialog Box ว่าต้องการใช้ Processor ตัวไหนในการทำงาน ให้เลือก PPC405_0 (มีสองตัวคือ 0 กับ 1 ตัว 0 จะเป็น default ของระบบ) จากนั้นใน tab JTAG Properties ให้เลือกสายสัญญาณที่จะส่งข้อมูลเป็น type USB และเลือกความเร็วเป็น 6000000 กด save แล้วระบบจะทำการเริ่ม XMD ขึ้นมา

ในระบบ XMD นี้เป็นการติดต่อไปยัง PowerPC Core ผ่านสาย USB ฉะนั้นระบบที่ต้องการติดต่ออยู่นี้จะมาจากตัวบอร์ดทดลองโดยตรงซึ่ง PowerPC ใช้ Embedded Linux เป็นระบบปฏิบัติการฝังตัวเอาไว้

```

G:\VEDK\bin\nt\xmd.exe
INFO:MDT - Assumption: Selected Device 3 for debugging.
JTAG chain configuration
-----
Device  ID Code      IR Length  Part Name
  1      05059093      16         XCF32P
  2      0a001093       8        System_ACE
  3      0127e093      14         XC2VP30

XMD: Connected to PowerPC target. Processor Version No : 0x200108a0
Address mapping for accessing special PowerPC features from XMD/GDB:
  I-Cache <Data> : Disabled
  I-Cache <Tag>  : Disabled
  D-Cache <Data> : Disabled
  D-Cache <Tag>  : Disabled
  ISOCM         : Start Address - 0xffff8000, Size - 32768 bytes
  TLB           : Disabled
  DCR           : Disabled

Connected to "ppc" target. id = 0
Starting GDB server for "ppc" target (id = 0) at TCP port no 1234
XMD% ls
TestApp_Memory  implementation  ppc405_0      system.log    system_incl.make
__xps           libgen.log     ppc405_1      system.make
data            pcores        synthesis     system.mhs
etc             platgen.log    system.bsh    system.mss
hdl             platgen.opt    system.gui    system.xmp
XMD% cd TestApp_Memory
XMD% ls
executable.elf  src
XMD% doe executable.elf
  
```

รูปที่ 4.35 หน้าต่าง XMD แสดงสถานะ shell prompt

เมื่อพบ Shell Prompt แล้วให้พิมพ์ cd TestApp_Memory ตามด้วย dow executable.elf เมื่อดาวน์โหลดเสร็จแล้วให้กด run เพื่อทำการรันโปรแกรม

```

G:\EDK\bin\nt\xmd.exe
invalid command name "doe"
XMD% dow executable.elf
WARNING: Attempted to read location: 0xffffffffc. Reading ISOCM memory not supported
section, .text: 0xffff8000-0xffff849c
section, .init: 0xffff849c-0xffff84c0
section, .fini: 0xffff84c0-0xffff84e0
section, .boot0: 0xffff84e0-0xffff84f0
section, .boot: 0xffffffffc-0x00000000
section, .rodata: 0x21800000-0x21800032
section, .sdata2: 0x21800034-0x21800034
section, .sbss2: 0x21800034-0x21800034
section, .data: 0x21800038-0x21800030
section, .got: 0x21800030-0x21800030
section, .got1: 0x21800030-0x21800030
section, .got2: 0x21800030-0x21800034
section, .ctors: 0x21800034-0x21800034
section, .dtors: 0x21800034-0x21800035
section, .fixup: 0x21800035-0x21800035
section, .eh_frame: 0x21800035-0x21800036
section, .jcr: 0x21800036-0x21800036
section, .gcc_except_table: 0x21800036-0x21800036
section, .sdata: 0x21800036-0x21800036
section, .sbss: 0x21800036-0x21800036
section, .bss: 0x21800036-0x21800038
section, bss_stack: 0x21800038-0x218000b9
section, bss_heap: 0x218000b9-0x21800139
Downloaded Program executable.elf
Setting PC with program start addr = 0xffffffffc
PC reset to 0xffffffffc, Clearing MSR Register
XMD% run
PC reset to 0xffffffffc, Clearing MSR Register
Processor started. Type "stop" to stop processor
RUNNING>

```

รูปที่ 4.36 หน้าต่าง XMD แสดงสถานะการ run ของ program อยู่

ในตอนนี้ให้เปิด Hyper Terminal ขึ้นมาใหม่ ให้ตั้งค่าดังนี้

Bits per Second= 115200

Data Bits = 8

Parity = None

Stop Bits = 1

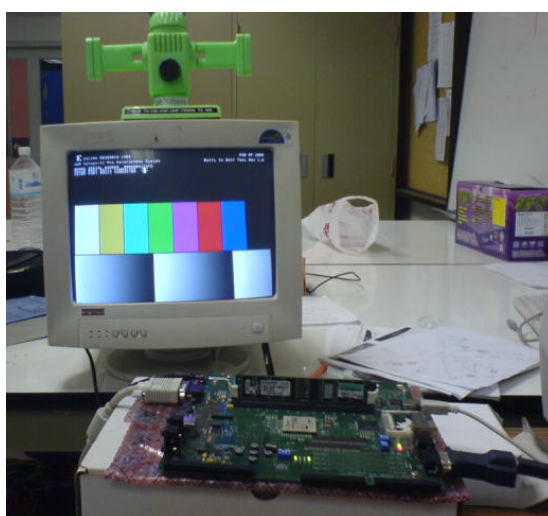
Flow Control = None

แล้วกดปุ่ม reset ที่บอร์ดทดลองจากนั้น ก็จะได้ข้อความว่า “Enter Main, Exit Main”

ขึ้นที่หน้าจอ terminal เป็นอันเสร็จสิ้น ซึ่งสามารถสร้าง Embedded Processor ขึ้นมาควบคุมการทำงานของ FPGA ได้

การพัฒนาอัลกอริทึม Superimposed Text detection ลงบนบอร์ด FPGA

เริ่มต้นนั้นจะเป็นการแสดงผลภาพ Bitmap ออกมาหน้าจอมอนิเตอร์ คือในขั้นตอนแรกนั้นจะทดสอบโดยการโหลดเอาไฟล์ภาพ Bitmap เข้าไปเก็บไว้ใน DDR-SDRAM แล้วให้แสดงผลภาพออกจากหน้าจอมอนิเตอร์ ซึ่งจะต้องมีการเพิ่มอุปกรณ์คือ DDR SDRAM ขนาด 512 MB และ Compact Flash ขนาด 1GB ของ Kingston จอมอนิเตอร์ที่รองรับการแสดงผลได้ 800*600 เป็นอย่างน้อย

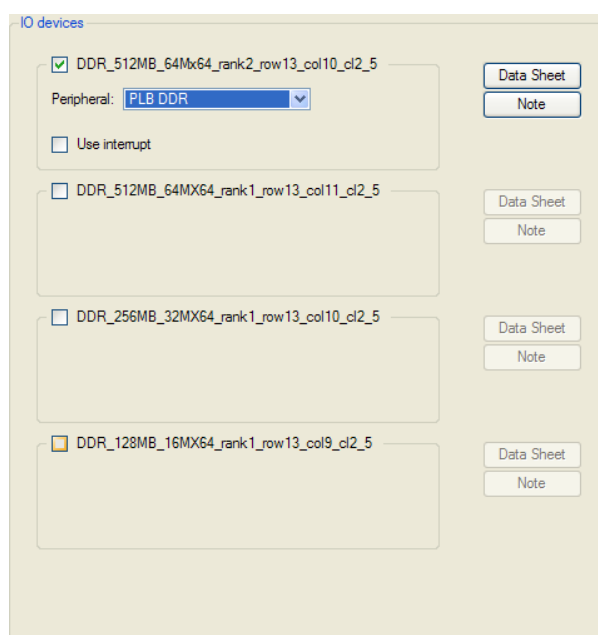


รูปที่ 4.37 รูปการต่ออินเตอร์เฟสระหว่าง จอภาพ, บอร์ด, แรม และ การ์ด



รูปที่ 4.38 รูปการต่ออินเตอร์เฟสระหว่าง จอภาพ, บอร์ด, แรม และ การ์ด (ต่อ)

เริ่มจากการสร้าง Project Base System Builder กันใหม่ครับ ทำตามขั้นตอนเดิมในบทที่แล้วมาจนถึงขั้นตอน Processor Configuration ให้ท่านเลือก Device ดังต่อไปนี้มาใช้งาน RS232, SysACE Compact Flash, DDR_512MB_64Mx64_rank2_row13_col10_cl2_5, Plb_bram_if_ctrl_1 (ให้เลือกขนาดของ BlockRam ไว้ 64 MB) จากนั้นให้เลือก StdIO ด้วย RS232 ส่วนโปรแกรมที่ไว้ทดสอบนั้น ในคราวนี้จะทำการเว้นว่างไว้ทั้งสองช่อง เพราะต้องการจะสร้าง Software Project



รูปที่ 4.39 หน้าจอตัวเลือกขนาดของ RAM ที่จะใช้งานในระบบ

สำหรับการที่จะแสดงภาพไปออกยังหน้าจอมอนิเตอร์นั้น จำเป็นต้องใช้ Library Core เพื่อทำการควบคุมสัญญาณที่จะนำไปออกยังมอนิเตอร์ ที่มีชื่อว่า plb_tft_ctrl_ref_v1_00_d ซึ่งใน Library ชุดนี้ประกอบไปด้วยไฟล์ Verilog ทั้งหมดที่ควบคุมในส่วนการแสดงผลภาพให้ไปออกยังหน้าจอ โดยสามารถหาได้จาก โปรเจกต์ตัวอย่างที่ได้ดาวน์โหลดมาก่อนในโฟลเดอร์ PCores จะมีไฟล์ชุด plb_tft_ctrl_ref_v1_00_d อยู่ ให้ทำการก๊อปปี้ทั้ง folder มาลงไว้ใน PCores ของโปรเจกต์ และทำการก๊อปปี้ โฟลเดอร์ Drivers ที่อยู่ในโฟลเดอร์หลักของ Project Slider มาด้วยเพราะในนั้นจะมีโฟลเดอร์ tft_ref_v1_00_a ซึ่งเป็น reference สำหรับการใช้งานคำสั่งที่เกี่ยวกับการแสดงผลออกหน้าจอภาพที่ทาง Xilinx จัดมาให้ในภาษา C

ในขั้นตอนต่อไปจะทำการดัดแปลงไฟล์ 3 ไฟล์ คือ System.mhs = “Microprocessor Hardware Specification (MHS)” which describes the instantiations and connections of hardware

components. โดยที่ระบบ Simulation Model Generator จะอ่านค่าจากไฟล์ดังกล่าวเพื่อไปทำการ generate ชุด Netlist และ ไฟล์ Bistream สำหรับส่วนของ Hardware Core System.mss = ไฟล์ mss จำกัดความหมายของไดรเวอร์ที่ทำงานร่วมกับ peripheral , Standard I/O devices interrupt handler routines และซอฟต์แวร์ที่เกี่ยวข้องอื่นๆ Libgen จะทำการตรวจสอบไลบรารีและไดรเวอร์ได้จากไฟล์นี้

สำหรับการใช้งานไฟล์ mhs, mss สามารถอ่านรายละเอียดได้เพิ่มจาก Platform Specification Format Reference Manual ซึ่งอยู่ในโฟลเดอร์ Doc ของ EDK Installation Path, System.ucf = User Constraint Files ที่ขึ้นกับตัวบอร์ดทดลองที่จะใช้งาน เป็นไฟล์ที่เก็บค่าการ assign port และ netlist ที่จำเป็นจะต้องเชื่อมต่อกับตำแหน่ง port address ในบอร์ดทดลอง

ในขั้นตอนต่อไป จะอธิบายถึงไฟล์ system.ucf เมื่อตรวจสอบ Modules ที่มีการประกาศค่าเอาไว้ จะมี Module RS232_Uart_1 constraints , Module SysACE_CompactFlash constraints, Module DDR_512MB_64Mx64_rank2_row13_col10_cl2_5 constraints แต่ยังขาด Modules ในส่วนของ TFT LCD VGA framebuffer อยู่ ให้เติมค่า assign ต่อไปนี้ลงต่อไปได้เลย

```
# Module for VGA FrameBuffer TFT LCD
Net fpga_0_VGA_FrameBuffer_TFT_LCD_CLK_pin LOC=H12;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_CLK_pin IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_CLK_pin SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_CLK_pin DRIVE = 12;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_HSYNC_pin LOC=B8;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_HSYNC_pin IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_HSYNC_pin SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_HSYNC_pin DRIVE = 12;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_VSYNC_pin LOC=D11;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_VSYNC_pin IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_VSYNC_pin SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_VSYNC_pin DRIVE = 12;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<0> LOC=H15;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<0> IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<0> SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<0> DRIVE = 6;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<1> LOC=J15;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<1> IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<1> SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<1> DRIVE = 6;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<2> LOC=C13;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<2> IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<2> SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<2> DRIVE = 6;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<3> LOC=D13;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<3> IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<3> SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<3> DRIVE = 6;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin<4> LOC=D14;
```

[illegible]

```

Net fpga_0_VGA_FrameBuffer_TFT_LCD_R_pin<5> SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_R_pin<5> DRIVE = 6;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_BLNK_pin LOC=A8;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_BLNK_pin IOSTANDARD = LVTTTL;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_BLNK_pin SLEW = SLOW;
Net fpga_0_VGA_FrameBuffer_TFT_LCD_BLNK_pin DRIVE = 6;

```

ตั้งค่าไฟล์ mss

```

BEGIN DRIVER
  PARAMETER DRIVER_NAME = tft_ref
  PARAMETER DRIVER_VER = 1.00.a
  PARAMETER HW_INSTANCE = VGA_FrameBuffer
END

BEGIN DRIVER
  PARAMETER DRIVER_NAME = generic
  PARAMETER DRIVER_VER = 1.00.a
  PARAMETER HW_INSTANCE = opb2dcr_bridge_0
END

BEGIN LIBRARY
  PARAMETER LIBRARY_NAME = xilfatfs
  PARAMETER LIBRARY_VER = 1.00.a
  PARAMETER PROC_INSTANCE = ppc405_0
END

```

ในขั้นตอนถัดมานั้น ผู้พัฒนาจะอธิบายถึงการทำงานในแต่ละตัวแปรที่ได้ประกาศไว้ในตอนต้น ประกอบด้วย tft_ref จะเป็น Driver Reference สำหรับการทำงานของ VGA_FrameBuffer ตัวต่อมาก็คือสายเชื่อมส่งต่อสัญญาณจากชุด opb ไปยัง dcr_bridge ส่วนตัวที่ 3 คือ Library “xilfatfs” เป็นไลบรารีที่ควบคุมการ อ่าน/เขียนของไฟล์ที่ถูกเก็บอยู่ใน Xilinx System ACE compact flash

โค้ดภายในไฟล์ mhs ที่ได้สร้างขึ้น

```

PORT fpga_0_VGA_FrameBuffer_TFT_LCD_CLK_pin =
fpga_0_VGA_FrameBuffer_TFT_LCD_CLK, DIR = O
PORT fpga_0_VGA_FrameBuffer_TFT_LCD_HSYNC_pin =
fpga_0_VGA_FrameBuffer_TFT_LCD_HSYNC, DIR = O
PORT fpga_0_VGA_FrameBuffer_TFT_LCD_VSYNC_pin =
fpga_0_VGA_FrameBuffer_TFT_LCD_VSYNC, DIR = O
PORT fpga_0_VGA_FrameBuffer_TFT_LCD_BLNK_pin =
fpga_0_VGA_FrameBuffer_TFT_LCD_BLNK, DIR = O
PORT fpga_0_VGA_FrameBuffer_TFT_LCD_B_pin =
fpga_0_VGA_FrameBuffer_TFT_LCD_B, VEC = [5:0], DIR = O

```

```

PORT fpga_0_VGA_FrameBuffer_TFT_LCD_G_pin =
fpga_0_VGA_FrameBuffer_TFT_LCD_G, VEC = [5:0], DIR = 0
PORT fpga_0_VGA_FrameBuffer_TFT_LCD_R_pin =
fpga_0_VGA_FrameBuffer_TFT_LCD_R, VEC = [5:0], DIR = 0

BEGIN opb2dcr_bridge
  PARAMETER INSTANCE = opb2dcr_bridge_0
  PARAMETER HW_VER = 1.00.a
  PARAMETER C_BASEADDR = 0xD0000000
  PARAMETER C_HIGHADDR = 0xD00003FF
  BUS_INTERFACE SOPB = opb
  BUS_INTERFACE MDCR = dcr_v29_0
END

BEGIN plb_tft_cntlr_ref
  PARAMETER INSTANCE = VGA_FrameBuffer
  PARAMETER HW_VER = 1.00.d
  PARAMETER C_DEFAULT_TFT_BASE_ADDR = 0b000000000000
  PARAMETER C_PIXCLK_IS_BUSCLK_DIVBY4 = 0b1
  # start with display off
  PARAMETER C_ON_INIT = 0b0
  PARAMETER C_DCR_BASEADDR = 0b0000010000
  PARAMETER C_DCR_HIGHADDR = 0b0000010001
  BUS_INTERFACE MPLB = plb
  BUS_INTERFACE SDCR = dcr_v29_0
  PORT SYS_dcrClk = sys_clk_s
  PORT TFT_LCD_CLK = fpga_0_VGA_FrameBuffer_TFT_LCD_CLK
  PORT TFT_LCD_HSYNC = fpga_0_VGA_FrameBuffer_TFT_LCD_HSYNC
  PORT TFT_LCD_VSYNC = fpga_0_VGA_FrameBuffer_TFT_LCD_VSYNC
  PORT TFT_LCD_B = fpga_0_VGA_FrameBuffer_TFT_LCD_B
  PORT TFT_LCD_G = fpga_0_VGA_FrameBuffer_TFT_LCD_G
  PORT TFT_LCD_R = fpga_0_VGA_FrameBuffer_TFT_LCD_R
  PORT TFT_LCD_BLNK = fpga_0_VGA_FrameBuffer_TFT_LCD_BLNK
END

BEGIN dcr_v29
  PARAMETER INSTANCE = dcr_v29_0
  PARAMETER HW_VER = 1.00.a
  PARAMETER C_DCR_NUM_SLAVES = 1
END

```

ทำการเปลี่ยนแปลงค่า Address ใน Module ต่อไปนี้เพื่อให้ทำงานสอดคล้องกันได้ทั้งระบบ
ใน plb2opb_bridge ให้เพิ่ม code เข้าไปดังนี้

```

PARAMETER C_RNG0_BASEADDR = 0xD0000000
PARAMETER C_RNG0_HIGHADDR = 0xD0000FFF
PARAMETER C_RNG1_BASEADDR = 0x78000000
PARAMETER C_RNG1_HIGHADDR = 0x7807ffff
PARAMETER C_NUM_ADDR_RNG = 2

```

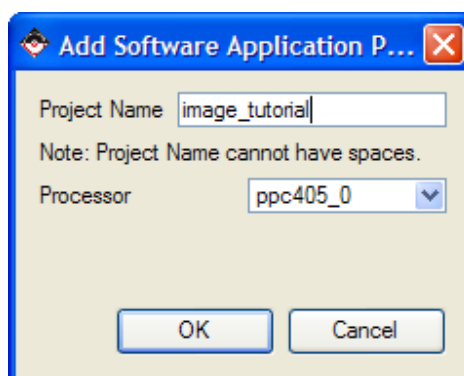
ใน opb_uartlite ให้เปลี่ยนค่าเป็นดังนี้

```
PARAMETER C_BASEADDR = 0x78050000
PARAMETER C_HIGHADDR = 0x7805ffff
```

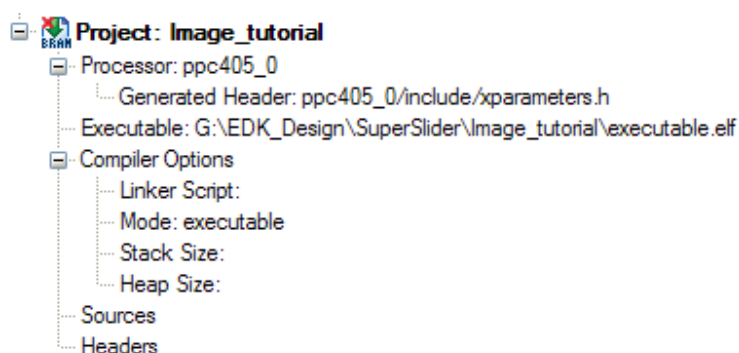
ใน opb_sysace ให้เปลี่ยนค่าเป็นดังนี้

```
PARAMETER C_BASEADDR = 0x78060000
PARAMETER C_HIGHADDR = 0x7806ffff
```

หลังจากนั้นให้ทำการสร้างไฟล์ Bitstream จากคำสั่ง Hardware->Generate Bitstream ซึ่งหลังจากที่ทำการสร้าง Bitstream สำเร็จแล้ว ให้ทำการสร้าง Software Project ขึ้นมา ในแท็บ Application ตัวเลือกแรกที่ท่านจะเห็นก็คือ Add Software Application Project... (ดังรูปที่ 4.40)



รูปที่ 4.40 หน้าจอการเพิ่ม Software Project ใหม่เข้าสู่ระบบ



รูปที่ 4.41 โปรเจ็ค Image_tutorial หลังจากได้ทำการเพิ่มเข้ามาสู่ระบบแล้ว

ให้คลิกขวาแล้วทำการสร้าง Software Project ชื่อว่า image_tutorial ขึ้นมา และให้เลือก Processor ที่จะใช้งานเป็น ppc405_0 เมื่อเสร็จแล้วให้ทำการ generate Linker Script โดยกำหนดให้ขนาดของ Stack กับ Heap ไว้เท่ากับ 0x2000 (เป็นค่าที่ใช้ในโครงการนี้)

หลังจากได้โค้ดแล้วก็ให้ทำการคอมไพล์ตัว Software ไปด้วยคำสั่ง Software->Build all user application เมื่อคอมไพล์เสร็จแล้วให้ทดสอบโปรแกรมที่ได้พัฒนาขึ้นมา

ในโครงการนี้ ได้ทำการอิมพลีเมนต์ อัลกอริทึม Superimposed Text Detection ที่ทีมผู้พัฒนาในคิดค้นขึ้น (กล่าวไว้ในตอนต้นของบท) ลงในบอร์ดทดลอง โดยยังคงใช้ภาพนิ่งเป็นภาพอินพุตจาก compact flash เนื่องจากการติดต่อวิดีโอไฟสวิตช์กับตัว FPGA ของบอร์ดทดลอง ต้องมีการสร้างตัวเล่นไฟสวิตช์โอในบอร์ดทดลองดังกล่าว ซึ่งมีขั้นตอนที่ซับซ้อน และ ยากต่อการพัฒนาเป็นอย่างมาก ดังนั้นทีมพัฒนาจึงเลือกส่วนของอัลกอริทึมลงบอร์ดทดลองเพียงอย่างเดียว